Customer Complaint Triage Graph: Beginner's Explainer

Topic 13 | Multi-Agent LangGraph Workflow | Plain English Concept Guide

1. The Real-World Problem: What Are We Trying to Solve?

Imagine running a massive online service. Every single minute, hundreds of customer complaints pour into your support inbox. Some customers were charged twice. Others have apps that keep crashing on Android. Some are sending praise, while others are furious and threatening legal action. If you hire hundreds of human agents to manually read, sort, and reply to every routine message, it costs millions and takes hours. If you use a single generic chatbot, it gets confused, makes false promises, and hallucinates.

Our Goal: Build an intelligent, automated digital support office. When a complaint arrives, the system instantly identifies who should handle it, checks the customer's past history, consults the company's official policy document, crafts an accurate reply, and immediately flags angry or dangerous complaints for human review.

2. Extreme Basics: What is LangGraph, and Why 'Graph'?

Traditional programs run in a straight, rigid line: Step 1 → Step 2 → Step 3. But real customer conversations don't work like that. Depending on what the customer says, you might need to jump to Billing, or Technical, or ask a clarifying question, or stop everything and alert a human manager.

LangGraph is a framework that lets us build AI programs shaped like a **flowchart (a graph)**. It has three core building blocks:

- **State (The Shared Clipboard):** A single dictionary or package of data that travels from worker to worker. It carries the customer's text, their history, the category, the policy facts, and the drafted reply.
- **Nodes (The Specialized Workers):** Each node is a focused function or agent designed to do exactly one job well (e.g., classify, look up policy, write a billing reply).
- **Edges (The Hallways and Doors):** The paths connecting the workers. A *conditional edge* is like a hallway with directional signs: it reads the clipboard and routes the ticket to the correct specialist door.

3. The Difference Between TA1 and TA2

System Dimension	TA1 (Baseline — What we started with)	TA2 (Extended — What we upgraded to)
Architecture	Single decision node with 3 basic reply branches.	Full Multi-Agent team with supervisor, memory, RAG, and human review.
Memory	Zero memory. Every complaint was treated as a stranger with no history.	**Two types of memory:** Short-term (chat turns) + Long-term (SQLite account history).
Company Policy	No policy access. Replies were generic AI guesses.	**RAG (Chroma DB):** Official policy FAQ retrieved and injected into answers.
Safety & Escalation	None. Angry threats received standard robotic replies.	**Human-in-the-Loop (HITL):** Anger and legal threats pause for human approval.
Reliability	If the AI failed or errored, the whole script crashed.	**Tool Failure Recovery:** Catches connection errors and sends safe fallback notes.

4. The Core Concepts Explained Like You're 10 Years Old

Here is what each component actually does behind the scenes, using simple daily-life analogies:

A. Short-Term Memory vs. Long-Term Memory (U3-1 & U3-2)

• **Short-Term Memory (The Active Phone Call Notepad):** When you call support, you tell the agent: *'I was double-charged $49.99 on invoice #8821.'* A minute later you ask: *'What was the invoice number I just gave you?'* If the agent has short-term memory (backed by **LangGraph SqliteSaver checkpointer**), they look at their notepad for this active call (keyed by thread_id) and answer: *'#8821'*. If you turn off the checkpointer, the agent suffers from **amnesia** and has no idea what you said five seconds ago.

• **Long-Term Memory (The Customer's Permanent File Folder):** When you hang up the phone, your notepad gets tossed, but the permanent record is saved to the company database (backed by **SQLite complaints.db**). Two months later when you call back from a different computer or after the server restarts, the agent looks up your Customer ID and sees: *'Ah, this customer has had 3 previous billing issues in September.'*

B. RAG — Retrieval-Augmented Generation (U3-3)

• **The Open-Book Exam Analogy:** An AI model like qwen2.5:3b is like a student who knows general English, but knows **nothing** about your specific company. Instead of forcing the AI to guess how long a refund takes (which causes hallucinations), we give it an open book: **our 10-chunk policy document**.
• **How Chroma and nomic-embed-text work:** We turn every policy paragraph into mathematical coordinates (embeddings). When a complaint mentions a duplicate charge, Chroma instantly pulls up Chunk #1 (which states: *'Duplicate charges are refunded within 5 to 7 business days'*) and hands it to the agent before it writes a single word.
• **The Grounded Fallback:** What if a customer asks: *'What is your stock ticker symbol on the NASDAQ?'* The policy book doesn't say anything about stock tickers. Because our prompt is grounded, the agent refuses to make up a fake stock symbol and truthfully replies: *'I don't know based on our documentation.'*

C. Self-Consistency Voting (Detecting Ambiguity Without Guessing)

Small 3B models struggle when you ask them: *'Are you sure you're right?'* (they get paranoid and doubt themselves). Instead, we use **majority voting**: we run the classification prompt 3 times at temperature=0.7 (slight randomness). If all 3 runs say *billing*, we are 100% confident. But if the runs split (e.g., 1 billing, 1 technical, 1 general), that disagreement itself is the proof that the complaint is ambiguous! No guessing required.

D. The Supervisor & Hop Limit (U4-2)

The **Triage Supervisor** is the air-traffic controller. It never talks directly to the customer; it only inspects the clipboard and points the arrow to the right specialist agent. To protect the system, we enforce a **Hop Limit (MAX_HOPS = 3)**. If a ticket bounces between departments 3 times without resolution, the supervisor halts the cycle and escalates it to human review, preventing infinite loops and runaway server costs.

E. Human-in-the-Loop Oversight (U4-3)

No company should let an autonomous bot reply to a customer who is screaming: *'I will sue you and contact my lawyer!'* Our system has an automated **anger and legal keyword detector**. When triggered, the system pauses execution using LangGraph's interrupt() mechanism. A human supervisor reviews the drafted reply and either enters **APPROVE** (sends it) or **REJECT** (intercepts it and assigns a human manager).

F. Tool Failure Recovery (U3-4)

In production, AI servers (like Ollama) occasionally time out, run out of memory, or drop connections. Without failure handling, the whole application crashes with an ugly error. In TA2, the classifier is protected by a try/except shield. If Ollama crashes, the graph catches the error and routes to a safe **failure_handler** node that sends a polite reassurance to the customer that their ticket was saved and will be reviewed manually.

5. Complete Project File Directory: Which File Does What Work?

If the jury points to any file in your project and asks: *'What is the purpose of this file?'*, here is your exact reference:

Folder / File Path	Purpose & What Work It Performs	Key Output / Role
ta1/ triage_graph.py	The **frozen baseline code** from your original TA1 submission. Contains the single classifier node and three basic response nodes with no memory or RAG.	Baseline reference point for evaluation comparisons.
ta2/ corpus/policy_faq.txt	The **company policy & FAQ document**. Contains 10 cleanly demarcated chunks covering refund timelines (5-7 days), app crash troubleshooting, login rules, and escalation SLAs.	The knowledge base that Chroma reads to answer customer questions.
ta2/ seed_db.py	The **SQLite database builder**. Creates complaints.db with two tables: profiles (long-term customer data) and complaints (past ticket log). Contains reset_db() to restore clean test data.	Provides persistent memory that survives server reboots.
ta2/ rag_setup.py	The **RAG vector database builder**. Reads policy_faq.txt, creates dense mathematical embeddings using nomic-embed-text, and stores them in chroma_db/.	Provides the retrieve(query, k) function used by agents.
ta2/ triage_graph_ta2.py	**The heart of the entire project!** Defines the full LangGraph state machine, connects all nodes and conditional edges, implements checkpointer short-term memory, specialist agents, anger detection, and human review.	The main executable script that runs the entire multi-agent workflow.
ta2/ requirements.txt	Lists all required Python libraries (LangGraph, LangChain, ChromaDB, SQLite checkpointer, Ollama integrations).	Ensures anyone can install the dependencies with one pip command.
ta2/ README.md	User guide containing prerequisites, model pull commands, and the **one-command execution line** required by the SOP.	Instructions for running both automated and interactive modes.
report/ report.pdf	The **official 4-page technical report** submitted to the jury. Contains the ASCII architecture diagram, 5-field agent role table, SOP checklist, TA1 vs TA2 evaluation, and challenge metrics.	Strictly budgeted to exactly 4 pages as mandated by the rubric.
report/ generate_pdf.py	The ReportLab script that compiles report.md into the pixel-perfect 4-page PDF with custom running headers and footers.	Automated PDF compiler.
traces/ *.txt (10 files)	Saved console outputs capturing evidence for every single grading item: TA1 baseline, TA2 full suite, memory with/without, restart demo, RAG k=1 vs k=3, tool failure recovery, HITL approved/rejected, and benchmark table.	Permanent proof for all jury criteria.

6. Step-by-Step: The Journey of a Customer Complaint

When you run the script, what actually happens to a complaint second-by-second? Here is the exact path:

- **Step 1: Input Arrives:** The customer submits: 'You charged me twice for my subscription this month, refund me immediately!'
- **Step 2: Check SQLite History:** The load_history_node checks SQLite for this customer ID. If they have past closed tickets, their profile is attached to the clipboard.
- **Step 3: Consult RAG FAQ:** The rag_node searches Chroma vector store and fetches Chunk #1: 'Duplicate charges are refunded within 5 to 7 business days'.
- **Step 4: Supervisor Classifies:** The classify_node runs 3 sampled votes at temperature=0.7. Result is 3:0 in favor of 'billing'. The hop counter increments to 1.
- **Step 5: Conditional Edge Routes:** The graph checks the category and sends the ticket to the billing_agent door.
- **Step 6: Specialist Writes Reply:** The billing_agent reads the customer complaint, their past history, and the 5-7 business day policy chunk, crafting an accurate, grounded reply.
- **Step 7: Safety & Anger Check:** The escalation_check scans the text. If no anger or legal threats are found, it bypasses human review. (If anger is found, it pauses for a human!).
- **Step 8: Checkpointer Saves Turn:** The SqliteSaver checkpointer records the turn under thread_id so any follow-up question will remember this interaction.
- **Step 9: Finished & Stored:** The final reply is returned to the user and saved to the permanent SQLite complaints table for future reference.

7. Jury Defense Cheat Sheet: How to Answer Like a Pro

Jury Q1: Show me the exact routing logic that performs the routing.

Answer: 'Our routing logic is implemented as a conditional edge in LangGraph called route_by_category(). It inspects the category field in the shared state. If the category is billing, it routes to billing_agent; if technical, to technical_agent; if needs_clarification, to the clarification agent; and if a tool exception occurred, to failure_handler. There is also a secondary conditional edge after the specialists that routes to human_review if the ticket was flagged as escalated.'

Jury Q2: What would you add to handle ambiguous complaints more gracefully?

Answer: 'Currently, when 3 sampled votes disagree, we route to needs_clarification_agent which asks a polite clarifying question. In production, we would add Parallel Multi-Intent Decomposition: when a customer has a compound complaint (like a failed payment combined with a website login crash), an intent-splitter node would split the ticket into two sub-tasks, run both Billing and Technical specialists in parallel, and merge their answers into one comprehensive response.'

Jury Q3: How do you prove that memory actually works?

Answer: 'We prove short-term memory through our amnesia test in traces/memory_with.txt versus traces/memory_without.txt. In Turn 1, the customer mentions invoice #8821 for $49.99. In Turn 2, they ask: what invoice did I just mention? With the checkpointer active, the agent recalls #8821. Without the checkpointer, the agent suffers amnesia and says it has no record of the invoice. For long-term memory, traces/restart_demo.txt proves that after killing the Python process, past complaints and customer profiles stored in SQLite are instantly reloaded.'

Jury Q4: Why did you use majority voting instead of asking the model if it is confident?

Answer: 'Small 3-billion parameter models like qwen2.5:3b suffer from meta-prompting bias. When you explicitly ask a small model are you sure you are right?, it tends to over-trigger and doubt itself even on obvious answers. By running self-consistency voting 3 times at temperature 0.7, disagreement between independent runs serves as the genuine mathematical signal for ambiguity.'